

Monster Type Recognition

A comparison of k-NN, neural network, SVM and Naive Bayes classifiers on the Kaggle dataset “Ghouls, Goblins, and Ghosts... Boo!”

1. Introduction

This report studies four classical machine-learning classifiers on the Kaggle dataset “Ghouls, Goblins, and Ghosts... Boo!”. The task is to recognise the type of a monster (Ghoul, Goblin or Ghost) from five features: `bone_length`, `rotting_flesh`, `hair_length` and `has_soul`, which are continuous values in $[0, 1]$, and `color`, which is a discrete feature with six values. The training set contains 371 labelled samples and the test set contains 529 samples whose labels are known only to Kaggle.

The four methods required by the assignment were implemented and compared:

- **k-Nearest Neighbors** with Euclidean distance ($k = 1, 3, 5, 10$);
- **Neural networks** with sigmoid hidden layers, a softmax output layer and Stochastic Gradient Descent (one and two hidden layers);
- **Support Vector Machines** with a linear and a Gaussian RBF kernel, using the one-versus-rest strategy;
- **Naive Bayes** with a normal distribution for the continuous features and a multinomial distribution for the color.

1.1 Evaluation

Because the test labels are hidden, performance was measured in two ways:

- **Official accuracy (Kaggle).** The predictions of every model for the test set were written to a CSV file (columns `id`, `type`) and uploaded to Kaggle. All accuracies in the result tables of this report are these Kaggle scores.
- **Local validation.** The accompanying program also holds out 20% of the training set (stratified by class), trains each model on the remaining 80% and prints accuracy, F1 score, precision and recall on the held-out part. It then retrains the model on the whole training set to produce the Kaggle submission file.

The metrics are defined from the true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN), where P and N are the numbers of positive and negative samples. For the three-class problem they are computed per class and combined with a weighted average:

$$\text{Accuracy} = (TP + TN) / (P + N)$$

$$\text{Precision} = TP / (TP + FP) \quad \text{Recall} = TP / (TP + FN)$$

$$F1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$$

2. Data preparation

The data are loaded with pandas. The id column is only an identification number, so it is not used as a feature; it is kept only to write the submission files. The color feature is text with six possible values, so, as suggested by the assignment, it is replaced by a normalised number in $[0, 1]$, like the other four features:

clear	white	green	blue	black	blood
1/6	2/6	3/6	4/6	5/6	6/6

The class labels (Ghoul, Goblin, Ghost) are kept as text for k-NN, SVM and Naive Bayes. For the neural networks they are converted to the integers 0, 1, 2 with a LabelEncoder and converted back to the original names after prediction, so the submission files always contain the correct class names. The libraries used are pandas, NumPy, scikit-learn and TensorFlow/Keras.

3. Methods and results

3.1 k-Nearest Neighbors

A new monster is assigned to the class that is most common among its k nearest training samples, using the Euclidean distance on the five normalised features. The votes are weighted by the inverse of the distance (weights="distance"), so closer neighbors count more. The classifier is scikit-learn's KNeighborsClassifier and was evaluated for $k = 1, 3, 5$ and 10.

k	Kaggle accuracy
1	0.67485
3	0.66918
5	0.68620
10	0.72211

Table 1. k-NN results (Euclidean distance).

The accuracy depends clearly on k : small values (1, 3, 5) are sensitive to noisy and overlapping samples and give 0.669–0.686, while $k = 10$ gives the best result, 0.72211, because the decision is smoother.

3.2 Neural networks

Fully connected networks were built with Keras. The input layer takes the five features. The hidden layers use the sigmoid activation. The output layer has three neurons (one per class) with the softmax activation, so it returns the probability that the monster belongs to each class; the predicted class is the one with the highest probability. The networks are trained with Stochastic Gradient Descent (Keras defaults: learning rate 0.01, batch size 32) for 50 epochs, minimising the sparse categorical cross-entropy. Two families of models were trained:

- one hidden layer with $K = 50, 100$ or 200 neurons;
- two hidden layers with $(K1, K2) = (50, 25), (100, 50)$ or $(200, 100)$ neurons.

Hidden layers	Neurons	Kaggle accuracy
1	50	0.71345
1	100	0.71430
1	200	0.71921
2	(50, 25)	0.71475
2	(100, 50)	0.71518
2	(200, 100)	0.72121

Table 2. Neural network results (sigmoid hidden layers, softmax output, SGD, 50 epochs).

All six networks perform similarly (0.713–0.722). Wider networks are slightly better, and for each width the two-layer network is slightly better than the one-layer network. The best network, (200, 100), reaches 0.72121.

3.3 Support Vector Machines

An SVM separates two classes by nature, so the three-class problem is handled with the one-versus-rest strategy (scikit-learn's `OneVsRestClassifier`): three binary SVMs are trained, each separating one class from the other two, and the class whose SVM is the most confident is predicted. Two kernels were used:

- **Linear kernel** with penalty parameter $C = 10$ and balanced class weights.
- **Gaussian RBF kernel** with balanced class weights. The submitted model uses the default gamma ("scale"); the program additionally evaluates gamma = 0.1, 1 and 10 on the validation split.

Kernel	Settings	Kaggle accuracy
Linear	$C = 10$, balanced weights	0.73156
Gaussian RBF	gamma = scale, balanced weights	0.72400

Table 3. SVM results (one-versus-rest).

The linear SVM gives the highest accuracy of the whole study, 0.73156, slightly above the RBF SVM. This suggests that the three classes are already close to linearly separable in the five-dimensional feature space, so the more flexible RBF kernel brings no benefit.

3.4 Naive Bayes

The Naive Bayes classifier was implemented from scratch (scikit-learn's `GaussianNB` cannot model a multinomial feature). It applies Bayes' rule and assumes that the five features are independent given the class y :

$$P(y | x) \propto P(y) \cdot P(x_1 | y) \cdot P(x_2 | y) \cdot P(x_3 | y) \cdot P(x_4 | y) \cdot P(\text{color} | y)$$

- **Prior $P(y)$** : the fraction of training samples that belong to class y .
- **Continuous features (x_1 to x_4)**: normal distribution, with the mean and the unbiased variance of each feature estimated per class from the training set:

$$p(x_i | y) = 1 / \sqrt{2\pi\sigma^2} \cdot \exp(- (x_i - \mu)^2 / (2\sigma^2))$$

$$\mu = \Sigma x_i / n \quad \sigma^2 = \Sigma (x_i - \mu)^2 / (n - 1)$$

- **Color:** multinomial distribution, estimated from the counts $n(c, y)$ of each of the six colors in class y , with Laplace smoothing so that no probability is zero: $P(\text{color} = c | y) = (n(c, y) + 1) / (n(y) + 6)$.

To avoid numerical underflow the product of probabilities is computed as a sum of logarithms, and the predicted class is the one with the highest score.

Method	Kaggle accuracy
Naive Bayes (Gaussian + multinomial)	-

Table 4. Naive Bayes result.

4. Comparison and best method

Table 5 shows the best configuration of each method, and Figure 1 shows the Kaggle accuracy of every configuration that was submitted.

Method	Best configuration	Kaggle accuracy
k-NN	k = 10	0.72211
Neural network	(K1, K2) = (200, 100)	0.72121
SVM	Linear kernel, C = 10	0.73156
Naive Bayes	Gaussian + multinomial	-

Table 5. Best configuration of each method.

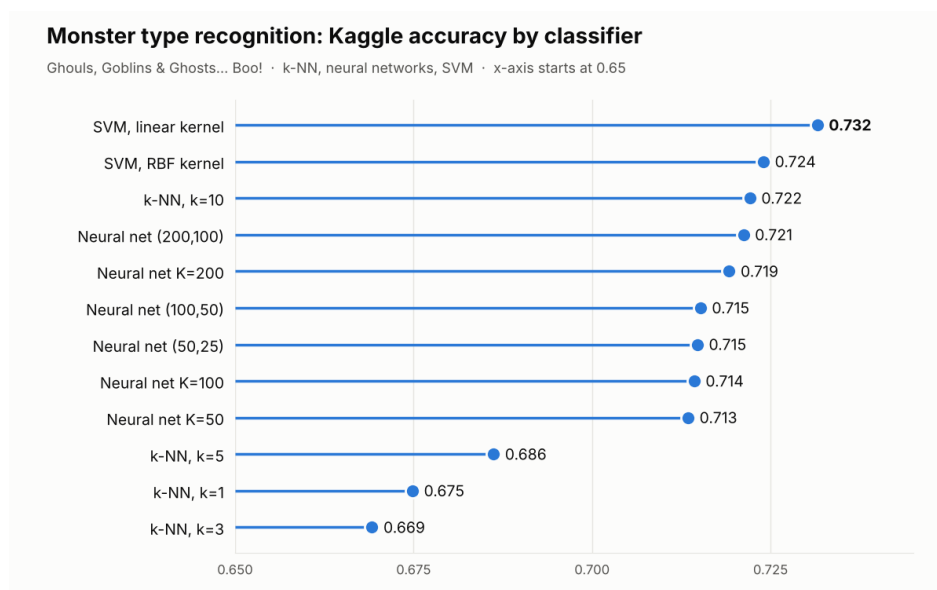


Figure 1. Kaggle accuracy of every submitted configuration (the x-axis starts at 0.65).

Best method. Among the methods evaluated, the Support Vector Machine with a linear kernel achieved the highest accuracy, 0.73156, followed by the RBF SVM (0.72400), k-NN with $k = 10$ (0.72211) and the (200, 100) neural network (0.72121).

All the tuned models lie within about one percentage point of each other, and the test set has only 529 samples, so a single sample changes the accuracy by about 0.19 percentage points. The differences between the top four configurations are therefore small and should not be over-interpreted. What is clear is that k-NN needs a large enough k , that the neural

networks need no more than a modest number of neurons, and that a simple linear separator is competitive with the more flexible models on this small dataset.